

2. Analyze the two GenAI programs.

- a. Name of two GenAI programs
 - i. Google Gemini flash 3.8 high
 - ii. Claude-Sonnet 4.6 (thinking)
- b. How you accessed them (e.g., Google query, CoPilot on my computer)
 - i. I used Google's AI harness Antigravity to access both models, again! It is really nice.
- c. Prompt used. (You must use the same prompt for both GenAIs.)
 - i. I had an AI take in the instructions, and concisely put them into a format for my other AI models to read:

///

Act as an expert C++ developer. Please write a complete, clean, object-oriented C++ program that solves Sudoku puzzles using recursive depth-first search with backtracking based on the requirements below.

Functional Requirements

1. **Input Files**: Process five input files named `puzzle1.txt`, `puzzle2.txt`, `puzzle3.txt`, `puzzle4.txt`, and `puzzle5.txt`.

- Each file contains a 9x9 Sudoku puzzle.
- Digits `1-9` represent pre-filled cells.
- Underscores (`\_`) represent empty/blank cells.

2. **Algorithm & Logic**:

- Implement **Depth-First Search (DFS) with Backtracking** recursively.
- For an empty cell, identify valid candidates (digits 1–9 that do not conflict with the same row, column, or 3x3 block).
- Recursively attempt placing candidates. If a dead end is reached, backtrack.
- **All Solutions**: Search exhaustively to find and print **ALL** valid solutions for a given puzzle (in case a puzzle has more than one solution). If no solution exists, print `"No solution found"`.

3. **Required Output Format**:

For each file (`puzzle1.txt` through `puzzle5.txt`), output:

- The puzzle file name (e.g., `puzzle1.txt`).
- The original puzzle board as read from the file.
- The solved board(s), or `"No solution found"` if unsolvable.

C++ Software Design Guidelines

- Use an **Object-Oriented Design** (e.g., a `SudokuBoard` or `SudokuSolver` class encapsulating the grid data and recursive logic).
- Include file I/O error handling (gracefully inform if a file cannot be opened).
- Follow modern C++ practices (`std::vector`, clear method responsibility, `const` correctness).
- Provide a `main()` function that loops through all five puzzle files sequentially.

- Write clean, self-documenting code with inline comments explaining the core recursive backtracking algorithm.

///

- d. Original Generated code from both GenAIs.
 - i. See folder generated_code for
 1. gemini_code.cpp
 2. claude_code.cpp
- e. Analyze the code in terms of correctness, execution time, space complexity, and maintainability as explained in the “GenAI Code Comparison” lecture.
 - i. Correctness
 1. Gemini’s program was able to do the given puzzles properly, and the output is incredibly helpful. If one of the 5 puzzles is invalid, it will say what line was wrong, what token was invalid, or whether the file just simply failed to load! It also uses recursion, 1 file for all the code, solves the puzzle, prints the solution(s), etc. The code comments and class descriptions are also much better than claude, who doesn’t use the special `/** */` class comments to define specifics. Gemini’s code also detects whether a board is invalid using the `isInitialBoardValid()` check.
 2. Claude’s program was able to do the puzzles given, and it prints out the solutions well formatted. Claude actually lists the number of solutions found, which I think is more correct than Gemini, but it dropped the word “file” from the output for the puzzle names, which I find to be a little less correct. The actual comments Claude left were in depth, but not as numerous and as well formatted as Gemini’s, as it doesn’t use the hinting format like Gemini’s. Claude is vulnerable to crashes though, since for an invalid token, Claude’s program catches the error and instantly terminates, stopping at that puzzle and not continuing to the others. Claude’s program also doesn’t check for an initially valid board, but it does tend to still produce “no solutions” for these invalid boards. However, it is not guaranteed!
 - ii. Execution Time / Time complexity
 1. Big O time complexity
 - a. Both programs have a maximum complexity of $O(1)$, since no matter what the actual program is bounded by a constant number of possibilities, empty cells, and size of the board. However, the actual complexity can be equated as something like $O(9^N)$ where N is the number of empty squares. Each empty square has 9 potential candidates, so 9^N is the worst possible scenario, so the actual variance depends on the number of empty squares. However, each program has essentially the same structure, as the depth-first searching with backtracking is implemented in the same way.

- b. Both programs' individual valid-placement steps are also $O(1)$ by definition, since each checks the 9 rows, the 9 columns, and the 9 sub-grid positions for possible placements.
- c. The only real difference between the two is the initial board-checking that Gemini does. The validation is initially slower, since it checks 81×27 possibilities for the board, but it does save time if a board is only found to be invalid after a lot of empty-spot placements. Claude runs through the DFS backtracking before eventually reaching the same conclusion (unless the board somehow has a valid existence despite an invalid board).

2. Execution time

- a. Empirical measurements across 60 benchmark runs (Windows, g++ MinGW-w64):
 - i. Unoptimized default build (-O0):
 - 1. Gemini: Mean = 13.18 ms, Median = 12.39 ms, Min = 9.75 ms
 - 2. Claude: Mean = 14.84 ms, Median = 13.09 ms, Min = 9.97 ms
 - 3. Gemini is ~11% faster out of the box.
 - ii. Optimized build (-O2):
 - 1. Gemini (-O2): Mean = 12.41 ms, Median = 11.36 ms, Min = 8.61 ms
 - 2. Claude (-O2): Mean = 16.07 ms, Median = 15.97 ms, Min = 8.84 ms
 - 3. Gemini is ~23% faster under -O2 optimization.
- b. Gemini appears to be faster on execution, and the reasoning is simply about caching and storing values vs finding them on-demand. In Claude's code, every iteration of "dfs()" runs nested loops over the grid and uses the accessor method `board_.get(row, col)`. In contrast, Gemini's `findEmptyCell()` scans and sets references directly by passing in references, not values, and `isValidPlacement()` operates directly on internal vectors without multiple getter layers. Also, Claude runs `std::cout` in the `dfs` method directly, which apparently causes flushing of the IO stream, and during recursive calls that can add a lot of overhead. Gemini saves its solutions until the end in `solutions_`, and then prints them all once the backtracking and recursion are done.

iii. Space Time / Space Complexity

- 1. For space complexity:
 - a. Recursive call stack depth: the maximum for both programs is 81 "frames", which are each a maximum of 64 bytes, but typically smaller. Both programs do this in a similar way, and the entire call

stack remains so small that each program is indistinguishable at this level.

- b. Both programs used `vector<vector<int>>` for their boards, which means their heap allocation remains the same, except for auxiliary memory. When Claude finds a solution, it prints the solution out, then overwrites that data and continues on. When Gemini finds a solution, it stores that grid as a full deep copy, meaning that for programs with a large number of solutions, Gemini's storage scales with the solutions in $O(n)$ where n is the number of solutions, but Claude's auxiliary memory is permanently $O(1)$.

2. Space time:

- a. For Space/time, which program is better depends entirely on what kind of sudoku puzzles the programs are solving. If the puzzles only have 1 to 2 solutions, then Gemini has a much better space/time complexity due to its faster execution times, since both have space requirements that are similar. However, when a puzzle has a lot of solutions, Claude wins in the space/time category due to how hi Gemini's auxiliary heap space overhead can get, since each solution is stored.

iv. Maintainability

1. Documentation:

- a. Gemini does a much better job at documentation, with proper class and function definition comments using the `@` symbols. It also just has more comments in general, making the code more maintainable from an outside perspective. Specifically, Gemini has a long comment block explaining the recursion, which is nice.

2. Separation of class controls:

- a. This is more semantic, but Gemini's `SudokuBoard` has a lot more of the functionality required for the search than Claude's implementation. Both used two classes, one of the board, and one for the search, but Gemini's `SudokuBoard` contains a lot of the methods that I would expect the `SudokuBoard` itself to contain, like checking whether a placement is valid through `isValidPlacement()`, and looking for an empty cell with `findEmptyCell()`. These methods make sense, at least to me, to have attached to the board class, not the search class, so I believe Gemini did a better job at using OOP principles.

3. Input possibility:

- a. Gemini has the ability to pass files in as arguments when the program is run through terminal, meaning you can add new puzzles to check without having to recode and recompile anything. Claude's puzzle set is hardcoded, so it would need to be refactored to accept any new puzzles, meaning Claude's code is

way less flexible and less testable, since you'd have to rewrite the program to test other files if you wanted to avoid editing any of the original puzzle files.

4. Error output:
 - a. Gemini's errors are handled leagues better. When an invalid token comes up, Gemini reports where that token is and then continues on with the other files, while Claude will error out completely, terminate, and maybe even crash.
- f. Select one to use as the basis for this assignment and justify why you picked it.
 - i. I would like to use Gemini's as a basis for my assignment. The reason I want to use Gemini's program is because it has more comments, so it will be easier for me to familiarize myself with. It also has better software architecture, meaning editing it will be easier since the coupling isn't as bad. It also already has advanced error handling, so I will not have to add extensive error handling to it, and can instead focus my efforts on improving spatial overhead and other problems.
- g. Explain what you did to improve it in terms of correctness, execution time, space complexity, and maintainability.
 - i. Correctness:
 1. For correctness, I added and corrected the output to include the number of solutions after the solutions are printed, like how the output appears in solution1.txt. For other things, like being able to use other files and detecting bad input, Gemini's program was already on top of it, so that final little output correction was really all I could do.
 - ii. Execution Time and Spatial Complexity:
 1. For execution time and spatial complexity, I actually managed to get both down at once! In Gemini's original program, the main spatial issue was that the solutions were deep copies of the SudokuBoard, which with many solutions added incredible overhead and a lot of dynamic heap allocations for memory, with each board having 240 bytes of vector headers and heap allocator metadata alone. Changing this to an array decreased page faults and cache misses because now the solutions are now 324 bytes each and all stored in a single continuous block of memory. This not only decreased spatial overhead since now the actual memory problems from the deep copies, like page faults and memory being scattered all over the place, do not occur anymore, but it also decreased execution time from a mean of 13.18 ms to 11.84ms! This is because memory is more strictly defined, meaning less searching and dynamic allocation.
 - iii. Maintainability:

1. I improved the program in terms of maintainability by adding inline comments EVERYWHERE. All lines are commented heavily and all functionality is clearly laid out, meaning that really anybody should be able to read the program and understand what is happening. Besides that, improving correctness is sort of difficult. The SudokuBoard contains all of the functions it should in my opinion, and the solver does its job perfectly. All large reused pieces of code are methods, and everything feels maintainable even for me.